

PoseCoach

Production Implementation Prompt Guide

YOLO26 Edition — v5.0

Upgraded from 9 to 10 Prompts — Thesis-Aligned

Previous Guide	Gaps Found	This Guide
9 prompts · v4	4 critical + 8 significant	10 prompts · thesis-ready

10 Prompts · YOLO26-Pose · FastAPI · React PWA · Gemini API · Docker · Pytest · Sentry

Table of Contents

1. What Changed: v4 → v5 Gap Analysis
2. 10-Prompt Build Plan
3. Thesis Timeline Alignment
4. PROMPT 01 — Dataset Preparation & YOLO26 Finetuning
5. PROMPT 02 — Infrastructure & YOLO26 Setup
6. PROMPT 03 — WebSocket + Inference Engine
7. PROMPT 04 — React PWA + Phone Camera
8. PROMPT 05 — RAG Chatbot Coach (Gemini Edition)
9. PROMPT 06 — Auth + Workout History
10. PROMPT 07 — Full Test Suite
11. PROMPT 08 — Observability
12. PROMPT 09 — Production Hardening & Deployment
13. PROMPT 10 — Thesis Evaluation Suite [NEW]
14. Production Prompt Engineering Tips

1. What Changed: v4 → v5

Before improving the prompts, the v4 guide was reviewed against the thesis proposal (PoseCoach_Proposal_v5.pptx). **4 critical gaps** and **8 significant improvements** were identified — all fixed in this guide.

Critical Gaps Found

#	Gap	Impact	Fix in v5
1	No thesis evaluation prompt	Cannot generate data for thesis evaluation chapter — SUS, chatbot accuracy, form variance all unmeasured	New Prompt 10 — Thesis Evaluation Suite
2	No user study infrastructure	SUS questionnaire with 10+ participants has no collection form, consent flow, or analysis script	Prompt 10 adds SUS deployment + analysis
3	Chatbot accuracy untested	Thesis claims ≥80% on 10 domain questions but no eval script exists	Prompt 10 adds eval_chatbot.py with verified Q&A; pairs
4	Form scoring variance unmeasured	Thesis claims <5% variance but no benchmark runs the same pose repeatedly	Prompt 10 adds eval_form_consistency.py

Significant Improvements

#	Change	Where	Why
1	Renamed GymVision → PoseCoach	All 10 prompts	Matches thesis proposal; avoids examiner confusion
2	Exercise scope documented	Prompt 03	7 of 22 Kaggle exercises supported — justification required for thesis
3	Fine-tuned model path fix	Prompt 02	MODEL_PATH now defaults to yolo26n-pose-posecoach.pt from Prompt 01
4	Privacy/ethics constraints	Prompts 02, 03, 09	JPEG frames never persisted; GDPR data deletion endpoint added
5	Session recording for user study	Prompt 04	Record button saves keypoint data for thesis analysis
6	RAG knowledge base spec strengthened	Prompt 05	Each file must include: errors, injury risks, cue phrases, angle ranges
7	Form variance test in CI	Prompt 07	test_form_score_variance asserts <5% — thesis metric in CI gate
8	Dataset statistics export	Prompt 01	CSV summary for thesis dataset description table

2. 10-Prompt Build Plan

The complete build sequence. Prompts must be executed in order — each depends on the previous. Prompt 10 is new in v5 and is the thesis-critical evaluation layer.

#	Name	What You Build	New in v5	Key Tech
01	Dataset Prep & Finetuning	Kaggle download, keypoint extraction, YOLO26 fine-tuning, mAP baseline	Dataset stats CSV export	Kaggle · YOLO26 · Ultralytics
02	Infrastructure & YOLO26	Docker, FastAPI, PostgreSQL, Redis, Alembic, CORS, model warmup	Privacy constraints, fine-tuned model path	Docker · FastAPI · Redis
03	WebSocket + Inference Engine	Real-time pose pipeline with frame queue, GPU mgmt, multi-person guard	Exercise scope docs, frame non-persistence	WebSocket · OpenCV · NumPy
04	React + Phone Camera	Full PWA with adaptive quality, reconnect, camera lifecycle	Session recording for user study	React · Vite · WebRTC · Canvas
05	RAG Chatbot Coach	Session-aware RAG with streaming SSE, conversation history, persistence	Strengthened knowledge base spec	ChromaDB · Gemini 2.0 Flash
06	Auth + Workout History	JWT auth, bcrypt, history API, progress charts, rate limiting	—	JWT · PostgreSQL · Recharts
07	Full Test Suite	Pytest, Vitest, Playwright E2E, YOLO mAP eval, CI gate	Form variance test added	Pytest · Vitest · Playwright
08	Observability	Structlog JSON logs, Sentry, /health/deep, Prometheus metrics	—	Sentry · Prometheus · structlog
09	Production Hardening	Security headers, NGINX TLS, prod Docker, CI/CD pipeline	Ethics/privacy section, GDPR endpoint	NGINX · Vercel · Render · GH Actions
10	Thesis Evaluation Suite	SUS study, chatbot eval, form variance, data export, thesis tables	Entire new prompt	SUS · LaTeX · matplotlib

3. Thesis Timeline Alignment

This maps the 10 prompts to the 6-month thesis timeline from the proposal. Each prompt has a recommended execution window.

Month	Phase	Prompts to Execute	Thesis Deliverable
1	Setup & Literature	Prompt 01 (Dataset)	Literature review draft + dataset description table
2	Core CV System	Prompts 02 + 03	Working inference pipeline, form scorer, angle calculator
3	Core CV + Frontend	Prompt 04	Mobile PWA with camera streaming and skeleton overlay
4	RAG + Auth	Prompts 05 + 06	Chatbot with RAG, user auth, workout history
5	Testing + Observability	Prompts 07 + 08 + 09	Full test suite, CI/CD, production deployment
6	Evaluation + Writing	Prompt 10	All thesis tables, SUS results, final submission

Key insight: Prompt 10 (Thesis Evaluation) should be started in Month 5, not Month 6. The user study requires recruiting 10+ participants, which takes time. Start building the evaluation infrastructure while finishing deployment.

PROMPT 01 — Dataset Preparation & YOLO26 Finetuning

This prompt bridges the gap between a working app and a defensible thesis. It downloads the Kaggle Workout/Exercises Video Dataset, extracts YOLO26 keypoints from every video frame, fine-tunes the model on exercise-specific data, and establishes a mAP baseline that proves the model works on real gym footage — not just COCO benchmarks.

Role & Context

You are a senior ML engineer. PoseCoach infrastructure from Prompt 02 is working. Before running the full test suite, you must prepare the Kaggle dataset and establish that YOLO26-Pose performs accurately on real gym exercise videos. This prompt covers dataset download, keypoint extraction, optional fine-tuning, and a mAP baseline evaluation – all of which feed directly into your thesis evaluation chapter.

Deliverables

1. `scripts/download_kaggle.py` — automated dataset setup:

- Install kaggle CLI: `pip install kaggle`
- Download: `kaggle datasets download hasyimabdillah/workoutfitnessvideo`
- Unzip into `data/kaggle/` preserving per-exercise folder structure
- Print dataset summary: total videos, exercise class counts, total frames
- **[NEW v5]** Export `data/dataset_summary.csv` with columns: `exercise`, `video_count`, `total_frames`, `mean_duration_s` — this becomes Thesis Table 1

2. `scripts/extract_keypoints.py` — batch keypoint extraction:

- Load YOLO('yolo26n-pose.pt'); iterate every video in `data/kaggle/`
- Per frame: decode JPEG, run `model.predict()`, save `results[0].keypoints.xyn` as numpy array
- Skip frames where no person detected (`results` empty); log skip count per video
- Save output to `data/keypoints/{exercise}/{video_name}.npy` — shape (N_frames, 17, 2)
- Log inference_ms per frame; report mean and p95 latency across dataset
- **[NEW v5]** Export `data/extraction_stats.csv`: `exercise`, `total_frames`, `detected_frames`, `detection_rate`, `mean_inference_ms`, `p95_inference_ms`

3. `scripts/prepare_yolo_dataset.py` — convert to YOLO pose format:

- Extract one frame every 5 frames per video (avoid near-duplicate frames)
- 80/20 train/val split stratified by exercise class
- Write `data/kaggle_pose.yaml` with train/val paths and `nc=1` (person class only)

4. `scripts/baseline_eval.py` — mAP baseline before fine-tuning:

- `model.val(data='data/kaggle_pose.yaml')` on pretrained `yolo26n-pose.pt`
- Print and save: `mAP@0.5`, `mAP@0.5:0.95`, `mean_inference_ms` — this is your Thesis Table 2
- Exit 0 if `mAP@0.5 >= 0.80`; exit 1 with clear message if below threshold
- **[NEW v5]** Save results to `data/baseline_results.json` for Prompt 10 export

5. `scripts/finetune_yolo.py` — fine-tune on Kaggle exercise data:

- `model = YOLO('yolo26n-pose.pt');` `model.train(data='data/kaggle_pose.yaml', epochs=50, imgsz=640, batch=16)`
- Save best weights to `models/yolo26n-pose-posecoach.pt`

- Re-run baseline_eval.py with fine-tuned weights; compare mAP before vs after — Thesis Table 3

6. Update app/main.py MODEL_PATH env var to point to fine-tuned weights: models/yolo26n-pose-posecoach.pt

Constraints

- Requires kaggle.json API token in ~/.kaggle/kaggle.json — document setup in README
- extract_keypoints.py must use run_in_executor — never block the async loop
- Fine-tuning is OPTIONAL if baseline mAP ≥ 0.85 — document the decision in thesis
- All scripts must be idempotent — safe to re-run without re-downloading or re-processing
- Do NOT modify the WebSocket inference pipeline — scripts are standalone evaluation tools only
- **[NEW v5]** All CSV/JSON exports must include a timestamp and model version for reproducibility

Success Criteria

- python scripts/download_kaggle.py completes and prints dataset summary
- extract_keypoints.py produces .npy files for every video with shape (N, 17, 2)
- baseline_eval.py prints mAP@0.5 ≥ 0.80 on Kaggle val set (pretrained model)
- If fine-tuned: fine-tuned mAP@0.5 $\geq$ baseline mAP (improvement documented)
- Mean inference_ms < 80ms CPU on Kaggle frames — ready to quote in thesis
- **[NEW v5]** data/dataset_summary.csv and data/extraction_stats.csv exist and are non-empty

Why dataset stats export matters for your thesis: Your examiner will ask 'how many frames did you train on?' and 'what was the detection rate per exercise?' Having these as CSV files means you can import them directly into your thesis tables without manual counting.

PROMPT 02 — Infrastructure & YOLO26 Setup

Foundation layer. Everything in the later prompts depends on this being solid. The v2 version adds Redis, Alembic migrations, CORS, model warmup, and structured error handling that were missing from the original.

Role & Context

You are a senior ML backend engineer. You are building PoseCoach – a real-time AI exercise form analyzer. Start from a blank folder. The app will stream JPEG frames from a phone camera via WebSocket to a FastAPI server running YOLO26-Pose inference, returning keypoints + form feedback.

Deliverables

1. **docker-compose.yml** — services: fastapi (port 8000), postgres:15, redis:7-alpine
2. **Dockerfile** — Python 3.11-slim, installs requirements.txt
3. **.env.example** — POSTGRES_URL, REDIS_URL, GEMINI_API_KEY, JWT_SECRET, MODEL_PATH, ALLOWED_ORIGINS, ENVIRONMENT
4. **requirements.txt** — ultralytics>=8.4.0, fastapi, uvicorn[standard], sqlalchemy[asyncio], asyncpg, alembic, redis[asyncio], opencv-python-headless, numpy, python-dotenv, structlog, slowapi
5. **app/main.py** — FastAPI app with:
 - CORSMiddleware: origins from ALLOWED_ORIGINS env var
 - GET /health — returns {status, version, environment}
 - GET /health/deep — checks DB, Redis, YOLO model loaded
 - Lifespan context manager that loads YOLO26 model on startup using: model = YOLO(MODEL_PATH) — stored in app.state.model
 - **[NEW v5]** MODEL_PATH defaults to 'models/yolo26n-pose-posecoach.pt' (fine-tuned from Prompt 01); falls back to 'yolo26n-pose.pt' (auto-downloads) if fine-tuned model not found
6. **app/db.py** — async SQLAlchemy engine with: pool_size=10, max_overflow=20, pool_timeout=30
7. **app/models.py** — Base + Users + WorkoutSessions tables
8. **alembic/** — initialized Alembic with async support; first migration: 0001_initial_tables.py
9. **app/cache.py** — Redis async client with get/set/delete helpers
10. **scripts/test_yolo.py** — load YOLO from MODEL_PATH, run on webcam frame, print JSON keypoints + inference_ms; verify NMS-free; exit 1 if model missing or person not detected

Constraints

- Do NOT add auth or rate-limiting in this step
- YOLO26 model loaded ONCE in lifespan, stored in app.state.model — never re-loaded per request; use yolo26n-pose.pt for dev, yolo26x-pose.pt for production GPU server

- All secrets from environment variables only — no hardcoded values
- Alembic env.py must use the same async engine as app/db.py
- Redis client must be initialized in lifespan and stored in app.state.redis
- ALLOWED_ORIGINS must default to ['http://localhost:5173'] in dev
- /health/deep must return 503 (not 200) if any dependency is down
- **[NEW v5]** JPEG frame bytes must NEVER be written to disk or logged — process in memory only, discard after inference (privacy requirement for thesis ethics chapter)

Success Criteria

- docker-compose up starts all 3 services with no errors
- GET /health returns {status:'ok', version:'1.0.0', environment:'dev'}
- GET /health/deep returns {db:'ok', redis:'ok', yolo:'ok'}
- alembic upgrade head runs without error on a fresh DB
- python scripts/test_yolo.py prints 17 keypoints JSON; exits with code 1 and clear error message if MODEL_PATH is wrong

Why these additions are production-critical: Redis is needed for WS session sharing across workers. Alembic prevents schema drift. Model in app.state prevents 3-5s cold starts. /health/deep returning 503 ensures broken servers don't get traffic from platform health checks.

PROMPT 03 — WebSocket + Inference Engine

The real-time core. YOLO26's NMS-free end-to-end inference removes a post-processing step that YOLO11 required — this is accounted for in the predictor.

Role & Context

You are a senior ML backend engineer continuing PoseCoach from Prompt 02. The Docker stack, FastAPI app, and Redis are running. The YOLO26 model is loaded in `app.state.model`. YOLO26 is NMS-free and end-to-end — no post-processing step is needed after `model.predict()`. Implement the real-time pose analysis pipeline as a WebSocket endpoint.

Deliverables

1. `app/inference/websocket.py` — WebSocket at `/ws/analyze`:

- Per-connection `asyncio.Queue(maxsize=2)`; if full, drop the frame (do NOT block) and return `{status:'frame_dropped'}` immediately
- Receive loop puts frames in queue; worker coroutine consumes queue
- Sends ping every 30s; closes connection if pong not received in 10s
- On disconnect: cancel worker task, flush queue, log session summary

2. `app/inference/predictor.py` — PosePredictor class:

- `predict(frame_bytes)` runs in executor: `await loop.run_in_executor(executor, self._predict_sync, frame)`
- YOLO26 NOTE: results are end-to-end NMS-free — access keypoints directly via `results[0].keypoints.xy` (no NMS call needed)
- YOLO26 NOTE: RLE-based keypoint localization gives sub-pixel accuracy — use `results[0].keypoints.xyn` for normalized coords
- Guard: if results empty, return `{status:'no_person_detected'}`
- Guard: if multiple persons, use highest-confidence detection
- Call `torch.cuda.empty_cache()` every 100 frames; log GPU memory every 500 frames

3. `app/inference/smooth.py` — KeypointSmoothing:

- `deque(maxlen=5)` per keypoint index
- Discard keypoints with `conf < 0.5` before smoothing
- Returns smoothed `[x, y, conf]` for all 17 keypoints

4. `app/analysis/angle_calculator.py`:

- `calculate_angle(a, b, c)` using `numpy.arctan2` — returns degrees
- `get_exercise_angles(keypoints, exercise)` -> `Dict[str, float]`

5. `app/analysis/form_scorer.py`:

- `ANGLE_RANGES` dict at top of file for each exercise
- `score_squat`, `score_deadlift`, `score_curl`, `score_bench`, `score_ohp`, `score_lunge`, `score_plank` — each returns `FormResult(score: int, cues: List[str], angles: Dict[str, float])`
- **[NEW v5]** `SUPPORTED_EXERCISES` constant listing all 7 exercises with a docstring explaining: 'PoseCoach supports 7 of 22 exercises from the Kaggle dataset. These 7 were selected because they (a) use clearly visible keypoints from a single camera angle, (b) have well-defined joint angle ranges in exercise science literature, and (c) cover the major compound and isolation movement patterns. The remaining 15 exercises either require

multi-view cameras, involve occluded keypoints, or lack standardised form criteria.'

6. `app/analysis/rep_counter.py` — RepCounter class:

- `deque(maxlen=30)` for primary angle history
- Peak detection with `min_height` and `min_distance` thresholds
- Returns `{rep_count, phase: 'up'/'down', primary_angle}`

7. Websocket response schema (return every frame):

```
{keypoints, angles, form_score, cues, rep_count, phase, inference_ms, status:  
'ok'/'no_person_detected'/'frame_dropped'}
```

Constraints

- Do NOT rewrite `db.py` or `models.py` from Prompt 02
- Do NOT call any NMS function — YOLO26 is end-to-end, NMS-free by design; adding NMS will break results parsing
- `inference_ms` must be measured and returned in every response
- NEVER call `model.predict()` from the async event loop directly — always use `run_in_executor` with `ThreadPoolExecutor(max_workers=2)`
- Frame queue MUST drop (not block) when full — use `put_nowait()`
- Angle range constants must be in `ANGLE_RANGES` dict, not inline
- Cue strings: max 8 words, plain English, no jargon
- All exceptions in `_predict_sync` must be caught and logged — never let an inference error close the Websocket
- **[NEW v5]** Raw JPEG frame bytes must be discarded immediately after inference — do NOT store, cache, or log frame data

Success Criteria

- `ws://localhost:8000/ws/analyze` accepts frames and returns valid JSON
- `inference_ms` < 80ms on CPU (yolo26n-pose), < 25ms on GPU (T4)
- Sending 30 frames with no person returns `no_person_detected`, not an exception
- Sending frames faster than inference causes `frame_dropped`, not memory growth
- After 100 frames, no OOM error and GPU memory is stable
- Keypoint coordinates use `.keypoints.xyn` — NOT `.boxes` (wrong attribute)

YOLO26 vs YOLO11 — critical differences: YOLO26 is NMS-free end-to-end. Adding NMS to a YOLO26 pipeline will break keypoint parsing. YOLO26 uses RLE-based keypoint localization for sub-pixel accuracy. yolo26n-pose is 43% faster on CPU than yolo11n-pose.

PROMPT 04 — React PWA + Phone Camera

The mobile frontend. v5 adds session recording for the thesis user study — participants can record keypoint data during their workout for later analysis.

Role & Context

You are a senior frontend engineer. PoseCoach backend from Prompts 01-03 is running at `http://localhost:8000`. Build a mobile-first React PWA (Vite + TypeScript) that opens the phone rear camera, streams frames over WebSocket, and displays real-time pose feedback. The app must work in a gym (poor WiFi, phone in portrait mode, screen locking between sets).

Deliverables

- 1. `public/manifest.json`** — PWA manifest: name: PoseCoach, display: standalone, orientation: portrait, theme_color: #1B5E20
- 2. `src/serviceWorker.ts`** — Workbox service worker: precache static assets; network-first for API calls
- 3. `src/hooks/useCamera.ts`:**
 - `getUserMedia({video:{facingMode:'environment',width:{ideal:1280}}})`
 - Mirror front camera: CSS transform `scaleX(-1)` on video + canvas
 - `visibilitychange` listener: `stopStream()` on hidden, `restartStream()` on visible
 - Returns: {videoRef, canvasRef, switchCamera, cameraReady, error, facingMode}
- 4. `src/hooks/useWebSocket.ts`:**
 - Connects to `WS_URL` env var (default: `ws://localhost:8000/ws/analyze`)
 - Sends {frame: base64JPEG, exercise: string} at adaptive FPS
 - ADAPTIVE QUALITY: measure roundTripMs per frame; if > 300ms reduce JPEG quality to 50; if > 600ms reduce capture to 320x240
 - AUTO-RECONNECT: exponential backoff — 1s, 2s, 4s, 8s, max 30s; show reconnecting badge
- 5. `src/components/SkeletonOverlay.tsx`:**
 - Canvas overlay, exact same size as video (ref-synced)
 - COCO 17-keypoint connections with color: green/orange/red by score
 - Applies `scaleX(-1)` transform when `facingMode === 'user'`
 - Renders nothing if `status === 'no_person_detected'`
- 6. `src/components/FormHUD.tsx`** — Score gauge (top-right), rep counter (top-left), phase indicator, max 3 cue cards sliding in from bottom, connection status badge
- 7. `src/components/ExerciseSelector.tsx`** — chip bar for 7 supported exercises
- 8. `src/components/PermissionScreen.tsx`** — handles `NotAllowedError`, `NotFoundError`, `NotReadableError` with specific user-facing instructions
- 9. [NEW v5] `src/hooks/useSessionRecorder.ts`:**
 - Record button in FormHUD toggles recording on/off
 - During recording: accumulate every WebSocket response (keypoints, angles, form_score, cues, rep_count,

inference_ms) into an in-memory array

- On stop: export as JSON file download — {exercise, duration_ms, frames: [...], summary: {total_reps, avg_score, errors_detected}}
- This data feeds directly into the Prompt 10 user study analysis
- Recording indicator (red dot) visible in FormHUD during active recording

10. vite.config.ts — configure PWA plugin (vite-plugin-pwa)

Constraints

- Use Vite + React 18 + TypeScript — NO Create React App
- Tailwind CSS only — no third-party component libraries
- Frames MUST be captured with requestAnimationFrame, not setInterval
- Max 15 FPS default — enforce with timestamp delta check
- All WebSocket logic in useWebSocket.ts — no inline new WebSocket()
- Video and canvas must share dimensions via ResizeObserver — NOT hardcoded pixel values
- Camera must be stopped when component unmounts — no memory leaks
- On iOS Safari: add playsInline attribute to video element (required)
- **[NEW v5]** Session recorder must NOT store raw JPEG frames — only keypoint data and scores (privacy constraint)

Success Criteria

- App is installable on Android/iOS from browser (PWA manifest works)
- Camera starts automatically on permission grant, defaults to rear
- When network is throttled to 'Slow 3G', app reduces quality and maintains < 2s feedback latency
- Locking screen and unlocking resumes the camera without refresh
- WebSocket disconnect auto-reconnects within 30s
- No console errors on iOS Safari 16+
- **[NEW v5]** Record button produces a downloadable JSON file with all session data after stopping

PROMPT 05 — RAG Chatbot Coach [Gemini Edition]

The AI coaching layer. Uses Google Gemini API (free student tier via Google AI Studio). v5 strengthens the knowledge base specification to make the chatbot evaluation defensible in the thesis.

Role & Context

You are a senior backend and frontend engineer. PoseCoach from Prompts 01-04 is working – camera streams, YOLO26 runs, form scores display. Add the AI coaching chatbot using Google Gemini API (free tier) with RAG, session awareness, SSE streaming responses, and conversation memory.

Deliverables

1. app/chatbot/router.py — GET /chat/stream (Server-Sent Events):

- Query params: question, exercise, rep_count, avg_form_score, current_errors, conversation_id
- Returns SSE stream: data: <token>\n\n for each chunk
- Final event: data: [DONE]\n\n with {sources: []}
- Use FastAPI StreamingResponse with media_type text/event-stream

2. app/chatbot/rag.py — RAG pipeline:

- ChromaDB with persist_directory from CHROMA_PATH env var
- Embed with sentence-transformers/all-MiniLM-L6-v2
- top_k=5, similarity_threshold=0.70
- Log retrieved chunk scores at DEBUG level

3. app/chatbot/llm.py — Gemini API with streaming:

- Model: genai.GenerativeModel(model_name='gemini-2.0-flash', system_instruction=...)
- Build prompt injecting session context BEFORE the question
- Stream: for chunk in response: yield chunk.text
- Fallback on GoogleAPIError: yield 'I am having trouble connecting right now. Here is a tip: focus on controlled movement and proper breathing.'

4. [IMPROVED v5] app/chatbot/knowledge/ — 8 knowledge files (min 600 words each):

squat_technique.txt, deadlift_technique.txt, bench_technique.txt, curl_technique.txt, ohp_technique.txt, lunge_technique.txt, plank_technique.txt, programming_principles.txt

Each file MUST include all 5 sections:

- CORRECT FORM: Step-by-step technique description with joint angle ranges
- COMMON ERRORS: At least 5 named errors with descriptions (must match cue strings from form_scorer.py)
- INJURY RISKS: Specific injuries caused by each common error
- CUE PHRASES: Coaching cues a trainer would say (e.g. 'drive through your heels', 'chest up')
- PROGRAMMING: Sets, reps, rest periods for beginner/intermediate/advanced

This structure ensures the chatbot can answer the 10 evaluation questions in Prompt 10 accurately.

5. app/chatbot/ingest.py — chunk (size=600, overlap=100) and embed all knowledge files into ChromaDB; idempotent (skip if exists)

6. src/components/ChatDrawer.tsx:

- useEventSource hook for SSE streaming; append tokens to UI
- Conversation history stored in React state (last 10 exchanges)
- Session context badge showing current exercise + form score
- Wrapped in React.memo to prevent WS re-render interference
- 'PoseCoach Coach is typing...' indicator during streaming

Constraints

- Do NOT rewrite files from Prompts 01-04
- Use gemini-2.0-flash — NOT gemini-pro (deprecated) and NOT gemini-1.5-pro (uses paid quota)
- Add GEMINI_API_KEY to .env.example — NOT CLAUDE_API_KEY
- Streaming MUST use SSE not WebSocket for the chat endpoint
- ingest.py must be idempotent — safe to run multiple times
- CHROMA_PATH env var must default to './data/chromadb' if not set
- Conversation history must NOT be stored server-side (stateless backend)
- Rate limit: 10 req/min on /chat/stream to stay under Gemini free tier (15 req/min)

Success Criteria

- /chat/stream returns SSE tokens visible in the browser Network tab
- User sees first token appear within 800ms of submitting question
- After 3 exchanges, the bot references earlier messages correctly
- After docker-compose restart, ChromaDB still contains all chunks
- If GEMINI_API_KEY is missing, endpoint returns 200 with fallback message — not a 500 error
- Question about astrophysics returns 'I do not have enough information...' exactly
- 11th request within 1 minute returns 429 (rate limiter working)

PROMPT 06 — Auth + Workout History

Authentication and persistent history. Rate limiting on auth endpoints, proper DB connection pool config, and Alembic migrations for the new tables.

Role & Context

You are a senior backend and frontend engineer. PoseCoach from Prompts 01-05 is working. Now add JWT authentication, workout session persistence, and a workout history dashboard with progress charts.

Deliverables

1. app/auth/router.py:

- POST /auth/register — validate email + min 8-char password, hash with bcrypt, return 201
- POST /auth/login — verify credentials, return JWT (HS256, 24hr expiry)
- Rate limit: 10 req/min on /auth/login using slowapi

2. app/auth/middleware.py — get_current_user dependency: decode JWT from Authorization: Bearer header

3. app/history/router.py:

- POST /history — protected; body: {exercise, reps, sets, avg_form_score, duration_seconds}
- GET /history?limit=30&exercise=SQUAT — protected; reverse-chronological
- GET /history/stats — returns per-exercise trend data

4. alembic/versions/0002_add_workout_sessions.py — workout_sessions table with index on (user_id, exercise, created_at)

5. src/components/HistoryDrawer.tsx — past sessions list + Recharts LineChart for score trend

6. src/context/AuthContext.tsx — login/logout/register + JWT in httpOnly cookie (NOT localStorage — XSS risk)

Constraints

- JWT_SECRET from env var — never hardcoded
- Do NOT store JWT in localStorage — use httpOnly cookie with SameSite=Strict
- Passwords: minimum 8 characters, reject if email already exists
- Use Recharts only — no Chart.js, no D3
- DB queries must use SQLAlchemy async — no raw SQL strings
- Add Alembic migration 0002 — do NOT modify 0001 from Prompt 02

Success Criteria

- POST /auth/register + /auth/login work; login sets httpOnly cookie
- Unauthenticated GET /history returns 401
- 11th login attempt within 1 minute returns 429

- alembic upgrade head creates workout_sessions table
- GET /history/stats returns trend data after 3 saved sessions

PROMPT 07 — Full Test Suite

Comprehensive testing. v5 adds a form scoring variance test that directly maps to the thesis evaluation metric (<5% variance on repeated trials).

Role & Context

You are a senior engineer on PoseCoach. Prompts 01-06 are complete. Now add a comprehensive test suite covering the inference engine, API endpoints, React components, and end-to-end user flows.

Deliverables

BACKEND TESTS (pytest)

1. **tests/conftest.py** — fixtures: `async_client` (SQLite in-memory), `mock_yolo_model` (fixed 17-keypoint output), `sample_jpeg` (480x640)

2. **tests/test_angle_calculator.py**:

- `test_squat_correct_form`: `knee_angle 95 deg` gives score ≥ 85
- `test_squat_knees_caving`: `knee_angle 40 deg` triggers cue
- `test_calculate_angle_collinear`: edge case — 3 collinear points
- `test_all_exercises_score_range`: parametrize over all 7 exercises

3. **tests/test_websocket.py**: valid frame schema, empty frame `no_person`, fast frames `frame_dropped`

4. **tests/test_auth.py**: register/login flow, duplicate email, invalid token 401, rate limit 429

5. **tests/test_history.py**: save/retrieve session, stats trend calculation

6. **[NEW v5] tests/test_form_consistency.py**:

- `test_form_score_variance`: create 20 identical keypoint arrays (same pose), run through `form_scorer` 20 times, assert variance < 5%
- `test_form_score_deterministic`: same input must produce identical output (no randomness in scorer)
- Parametrize over all 7 exercises
- *This test enforces the thesis evaluation metric: form scoring consistency < 5% variance*

FRONTEND TESTS (Vitest)

7. **src/__tests__/useCamera.test.ts**: mock `getUserMedia`, test `switchCamera`, test `visibilitychange`

8. **src/__tests__/useWebSocket.test.ts**: mock `WebSocket`, test adaptive quality, test reconnect

E2E TESTS (Playwright)

9. **e2e/camera_flow.spec.ts**: grant camera, assert skeleton canvas, select exercise

10. **e2e/chat_flow.spec.ts**: type question, assert SSE response, assert typing indicator

YOLO ACCURACY EVAL

11. **scripts/eval_yolo.py**: run on 50-frame test video, mAP@0.5 gate (fail if < 0.80)

Constraints

- Backend tests must use SQLite in-memory — NOT the dev PostgreSQL DB
- YOLO model must be mocked in API tests — no real inference in unit tests
- All tests must pass with `pytest -x --timeout=30`
- Test coverage: target 80% for app/analysis/ — enforce with `pytest --cov=app/analysis --cov-fail-under=80`

Success Criteria

- pytest runs with 0 failures; app/analysis/ coverage $\geq 80\%$
- vitest run passes all frontend tests
- Playwright e2e passes both flows in headless Chrome
- **[NEW v5]** test_form_score_variance passes for all 7 exercises with variance $< 5\%$

PROMPT 08 — Observability

Structured JSON logging, Sentry error tracking, Prometheus metrics, and an enhanced health endpoint. Without this, you are flying blind in production.

Role & Context

You are a senior backend engineer. PoseCoach (Prompts 01-07) is working and tested. Now add full observability: structured JSON logging, Sentry error tracking, Prometheus metrics, and an enhanced health endpoint.

Deliverables

1. **app/logging_config.py** — structlog: JSON in prod, colored console in dev. Standard fields: timestamp, level, service, environment, request_id. Log every WS frame: {inference_ms, form_score, exercise, frame_dropped, user_id}. Replace ALL print() calls.
2. **app/middleware/request_id.py** — UUID request_id per request, X-Request-ID header, bind to structlog context
3. **Sentry integration** — sentry_sdk.init(dsn=SENTRY_DSN, traces_sample_rate=0.1); capture exceptions + slow transactions (> 2s)
4. **app/metrics.py** — Prometheus: ws_active_connections (Gauge), ws_frames_received_total (Counter), ws_frames_dropped_total (Counter), inference_duration_seconds (Histogram), form_score_distribution (Histogram), chat_requests_total (Counter)
5. **GET /metrics** — restricted by METRICS_TOKEN env var
6. **Enhanced GET /health/deep** — add inference_loaded, model_name, redis_ping_ms, db_query_ms, uptime_seconds
7. **app/middleware/timing.py** — log slow requests (> 500ms) as warnings

Constraints

- SENTRY_DSN from env var — if missing, Sentry disabled silently
- /metrics restricted by Authorization header (METRICS_TOKEN)
- Logging must NEVER log: passwords, JWT tokens, JPEG frame bytes, or user PII beyond user_id
- structlog must be initialized before any other import in main.py
- Do NOT rewrite inference logic — add instrumentation only

Success Criteria

- docker-compose logs shows JSON-formatted logs in production mode
- After 10 WebSocket frames, GET /metrics shows non-zero values
- GET /health/deep returns inference_loaded: true and all timing fields

- `grep -r 'print(' app/` returns 0 results

PROMPT 09 — Production Hardening & Deployment

Security hardening, secrets management, NGINX TLS, production Docker, and CI/CD. v5 adds a GDPR-compliant data deletion endpoint for the thesis ethics chapter.

Role & Context

You are a senior DevOps and security engineer. PoseCoach (Prompts 01-08) is feature-complete and tested. Now harden it for production, configure NGINX with TLS, and set up CI/CD to deploy frontend to Vercel, backend API to Render, and GPU inference to Modal.com (free tier).

Deliverables

SECURITY

1. **app/middleware/security_headers.py** — X-Content-Type-Options: nosniff, X-Frame-Options: DENY, X-XSS-Protection, Referrer-Policy, Content-Security-Policy
2. **Tighten CORS** in app/main.py — production: only ALLOWED_ORIGINS; allow_methods=['GET','POST']
3. **DB-level input validation** on all text fields: max_length=500
4. **[NEW v5] app/auth/router.py — DELETE /auth/account:**
 - Protected endpoint — requires JWT
 - Deletes user record + all workout_sessions for that user_id
 - Returns 204 No Content on success
 - *Required for thesis ethics chapter: users must be able to delete all their data (GDPR Article 17 — right to erasure)*

NGINX

5. **nginx/nginx.conf** — HTTPS on 443, TLS 1.2/1.3, HTTP→HTTPS redirect, WebSocket upgrade headers, rate limit 30 req/min per IP, gzip for JSON

DOCKER

6. **deploy/docker-compose.prod.yml** — 2 uvicorn workers, health check depends_on, SSL cert volumes, watchtower with --label-enable

CI/CD (GitHub Actions)

7. **.github/workflows/ci.yml** — on PR: pytest --cov, vitest run, docker build; block merge on failure
8. **.github/workflows/deploy.yml** — on push to main: run CI → alembic upgrade head → deploy Vercel + Render → GET /health/deep gate

SECRETS

9. GitHub Actions Secrets for: RENDER_DEPLOY_HOOK, MODAL_TOKEN_ID, MODAL_TOKEN_SECRET, SENTRY_DSN, JWT_SECRET, GEMINI_API_KEY. Document in README: never commit .env.prod to git.

Constraints

- Do NOT re-engineer any app logic — this prompt is infrastructure only
- TLS certificate: use Certbot/Let's Encrypt with auto-renewal cron
- deploy.yml MUST run all tests before any deploy step — no exceptions
- alembic upgrade head BEFORE docker up in deploy.yml
- watchtower must use --label-enable flag
- docker-compose.prod.yml must be separate from docker-compose.yml

Success Criteria

- curl -I https://yourdomain.com/health shows HTTPS + security headers
- WebSocket connects over wss:// without errors
- GitHub Actions: a PR with failing test blocks merge
- GET /health/deep on prod returns 200 with all checks passing
- 31st request/min from same IP returns 429 at NGINX level
- **[NEW v5]** DELETE /auth/account removes user + all sessions; subsequent GET /history returns 401

PROMPT 10 — Thesis Evaluation Suite [NEW IN v5]

This entire prompt is new in v5. It is the thesis-critical layer that generates all evaluation data, runs the user study, and exports results into thesis-ready tables and charts. Without this prompt, your thesis evaluation chapter has no data.

Role & Context

You are a senior ML engineer and research assistant. PoseCoach (Prompts 01-09) is deployed and production-ready. Now build the complete thesis evaluation suite: automated benchmarks for all 5 evaluation metrics from the thesis proposal, a user study deployment, and a data export pipeline that generates thesis-ready tables and figures.

The 5 Thesis Evaluation Metrics

Metric	Target	Method	Script
Pose Detection Accuracy	mAP@0.5 $\geq$ 0.80	YOLO26 val on Kaggle test set	scripts/eval_yolo.py (from P07)
Inference Latency	< 80ms CPU / < 25ms GPU	Per-frame timing on 200 frames	scripts/eval_latency.py
Form Scoring Consistency	< 5% variance	20 repeated trials per exercise	scripts/eval_form_consistency.py
Chatbot Accuracy	$\geq$ 80% correct	10 verified Q&A pairs scored by LLM judge	scripts/eval_chatbot.py
User Study	SUS $\geq$ 70	10+ participants, SUS questionnaire	scripts/eval_user_study.py

Deliverables

BENCHMARK SCRIPTS

1. scripts/eval_latency.py — inference latency benchmark:

- Load YOLO model from MODEL_PATH
- Run inference on 200 frames from Kaggle test set
- Measure per-frame inference_ms (excluding I/O)
- Report: mean, median, p95, p99, min, max
- Separate runs for CPU (yolo26n-pose) and GPU (yolo26x-pose) if available
- Save results to data/eval/latency_results.json
- Exit 1 if mean > 80ms on CPU or mean > 25ms on GPU
- Generate matplotlib box plot: latency_boxplot.png

2. scripts/eval_form_consistency.py — form scoring variance:

- For each of the 7 exercises:
 - Create a 'perfect form' keypoint array from ANGLE_RANGES midpoints

- Create a 'poor form' keypoint array from ANGLE_RANGES extremes
- Run each through form_scorer 20 times
- Calculate variance and standard deviation of scores
- Report per-exercise: mean_score, std_dev, variance_pct, max_deviation
- Save to data/eval/consistency_results.json
- Exit 1 if any exercise has variance >= 5%
- Generate bar chart: consistency_chart.png

3. scripts/eval_chatbot.py — chatbot accuracy evaluation:

- Define 10 verified Q&A pairs in data/eval/chatbot_test_set.json:
 - Q1: 'Why does my lower back hurt during deadlifts?'
 - Q2: 'How deep should I squat?'
 - Q3: 'What muscles does the overhead press work?'
 - Q4: 'How do I fix my knees caving in during squats?'
 - Q5: 'What is the correct elbow angle for bicep curls?'
 - Q6: 'How many sets and reps should a beginner do for bench press?'
 - Q7: 'What are the risks of arching your back during deadlifts?'
 - Q8: 'How long should I hold a plank?'
 - Q9: 'What is the difference between a front and back squat?'
 - Q10: 'How should I breathe during a lunge?'
- Each pair has: question, expected_answer_keywords (list of required concepts), verified_source (exercise science reference)
- Send each question to /chat/stream, collect full response
- Score each response: 1 if all expected keywords are present in the answer, 0.5 if partial, 0 if missing or hallucinated
- Report: per-question score + total accuracy percentage
- Save to data/eval/chatbot_results.json
- Exit 1 if accuracy < 80%
- Also test 2 out-of-domain questions to verify the 'I do not have enough information' guard

USER STUDY

4. scripts/deploy_sus_form.py — SUS questionnaire deployment:

- Generate a self-contained HTML file: public/sus_questionnaire.html
- Standard SUS 10-item questionnaire (Brooke, 1996) with:
 - Participant ID field (anonymous — no PII collected)
 - Likert scale 1-5 for each of the 10 SUS statements
 - Optional free-text feedback field
 - Submit button that downloads results as JSON file
- Include a participant consent statement at the top:

'This study evaluates the usability of PoseCoach. Your responses are anonymous. No personal data is collected. You may withdraw at any time.'
- The HTML file works standalone (no backend required) — participants open it in a browser after testing PoseCoach

5. scripts/eval_user_study.py — SUS score calculation:

- Read all participant JSON files from data/eval/sus_responses/
- Calculate SUS score per participant using standard formula:
 - For odd items (1,3,5,7,9): score = response - 1
 - For even items (2,4,6,8,10): score = 5 - response
 - SUS score = sum × 2.5
- Report: per-participant SUS score, mean SUS, std_dev, min, max

- Save to data/eval/sus_results.json
- Exit 1 if mean SUS < 70
- Generate bar chart: sus_scores_chart.png

THESIS DATA EXPORT

6. scripts/export_thesis_tables.py — the master export script:

- Read ALL eval results from data/eval/*.json + data/*.csv (from Prompt 01)
- Generate the following thesis-ready outputs in data/thesis/:

Table 1: thesis_table_1_dataset_summary.csv — exercise, video_count, frames, detection_rate

Table 2: thesis_table_2_baseline_map.csv — model, mAP@0.5, mAP@0.5:0.95, mean_inference_ms

Table 3: thesis_table_3_finetuned_comparison.csv — pretrained vs fine-tuned mAP side-by-side

Table 4: thesis_table_4_latency_benchmark.csv — device, mean, median, p95, p99

Table 5: thesis_table_5_form_consistency.csv — exercise, mean_score, std_dev, variance_pct

Table 6: thesis_table_6_chatbot_accuracy.csv — question, score, response_excerpt

Table 7: thesis_table_7_sus_results.csv — participant_id, sus_score

Figure 1: thesis_fig_1_latency_boxplot.png

Figure 2: thesis_fig_2_consistency_chart.png

Figure 3: thesis_fig_3_sus_scores.png

Figure 4: thesis_fig_4_map_comparison.png (pretrained vs fine-tuned bar chart)

- Also generate LaTeX table snippets in data/thesis/latex/ for direct copy-paste into thesis

7. data/eval/chatbot_test_set.json — the 10 verified Q&A pairs with expected keywords and source references

Constraints

- Do NOT modify any application code from Prompts 01-09 — evaluation scripts are standalone
- SUS questionnaire must use the EXACT 10 standard SUS items (Brooke, 1996) — do not modify wording
- Participant consent statement is required — no PII collection; anonymous participant IDs only
- eval_chatbot.py must call the ACTUAL /chat/stream endpoint (not mock) — this tests the real RAG pipeline
- All charts must use matplotlib with a consistent style: figsize=(10,6), tight_layout, sans-serif font
- LaTeX table exports must use booktabs style (\toprule, \midrule, \bottomrule) — no vertical lines
- All eval scripts must be runnable independently: python scripts/eval_latency.py (no dependencies on other eval scripts)
- export_thesis_tables.py must gracefully handle missing eval results — warn but do not crash if a metric has not been run yet

Success Criteria

- python scripts/eval_latency.py completes and reports mean < 80ms CPU
- python scripts/eval_form_consistency.py passes with < 5% variance for all 7 exercises
- python scripts/eval_chatbot.py scores >= 80% on the 10 verified questions
- public/sus_questionnaire.html opens in a browser and produces a downloadable JSON on submit
- python scripts/eval_user_study.py calculates SUS scores correctly from sample data

- python scripts/export_thesis_tables.py generates all 7 CSV tables + 4 PNG figures + LaTeX snippets
- data/thesis/ directory contains everything needed for the thesis evaluation chapter

Running the full evaluation pipeline:

1. python scripts/eval_yolo.py # mAP gate (from Prompt 07)
2. python scripts/eval_latency.py # Inference latency benchmark
3. python scripts/eval_form_consistency.py # Form scoring variance
4. python scripts/eval_chatbot.py # Chatbot accuracy
5. [Run user study with 10+ participants]
6. python scripts/eval_user_study.py # SUS score calculation
7. python scripts/export_thesis_tables.py # Generate all thesis tables + figures

14. Production Prompt Engineering Tips

Lessons from the gap analysis — what made the original prompts good for a prototype but insufficient for production and a thesis submission.

1 — Split prompts that cross architectural boundaries

The original Prompt 05 combined JWT auth + workout history + NGINX + Docker + CI/CD in a single pass. These are four completely different concerns. When a single prompt crosses too many layers, the model makes tradeoffs that create gaps at each boundary. The rule: one concern per prompt, one success criteria per layer.

2 — Specify failure modes, not just happy paths

Every constraint has a corresponding failure mode it prevents. 'Drop frames when queue full' prevents memory growth. '/health/deep returns 503' prevents broken servers getting traffic. 'Fallback message if Gemini API fails' prevents 500 errors reaching users. For production systems, the constraint list should be as long as the deliverables list.

3 — Name the exact error classes to handle

The original camera permission screen handled a generic error. The v5 version handles `NotAllowedError`, `NotFoundError`, and `NotReadableError` — each with different user instructions. Naming the exact exception class forces the model to write specific handling, not a catch-all.

4 — Add a model quality gate

Real CV apps degrade silently — a new YOLO model version might have better benchmark mAP but worse performance on your specific dataset (low-light gym footage). The `eval_yolo.py` script with `exit 1` in CI is the only way to catch this before it reaches users.

5 — State security defaults explicitly in every prompt

The `httpOnly` cookie constraint, `ALLOWED_ORIGINS`, and `METRICS_TOKEN` are all things a model will skip if you do not state them. Security-by-default must be in the constraints, not left as 'assumed best practice'.

6 — [NEW] Build evaluation infrastructure alongside features

The biggest gap in v4 was that all evaluation was deferred to 'the thesis writing phase.' By then it is too late — you discover that your chatbot answers 60% correctly but you have no time to improve the knowledge base. Prompt 10 should be started in Month 5, not Month 6.

7 — [NEW] Map every thesis claim to a script with `exit 1`

Your thesis proposal makes 5 quantitative claims. Each one must have a script that (a) measures the metric, (b) compares against the target, and (c) exits 1 if the target is missed. If your CI pipeline passes, your thesis claims are defensible. If it fails, you know exactly what to fix.

8 — [NEW] Export data in thesis-ready formats from the start

Do not plan to 'manually create tables later.' By Month 6 you will be exhausted from debugging deployment. Every eval script should write a CSV or JSON that `export_thesis_tables.py` can consume directly. The LaTeX snippets are not optional — they save days of formatting.